

Proyecto: VALGREEN — Sistema de Gestión de Repostería

Documento: Integracion

Generado el: 2026-09-23 16:28

PROMPT 8 — Pruebas de integración

Proyecto: VALGREEN — Sistema de Gestión de Repostería **Stack:** PHP 8.2+, Laravel 11, MySQL 8, Blade + Bootstrap 5

Componentes: Rutas (web.php), Controladores (8), Modelos (14), Validaciones (\$request->validate), Middleware (auth + RoleMiddleware), Autenticación (Laravel Auth + Hash), Vistas (20+ blade).

Flujos de integración (INT-001 a INT-010)

INT-001 Ciclo completo Auth + Roles multi-dashboard

- **ID:** INT-001
- **Flujo:** Login Admin → Dashboard Admin → Logout → Login Vendedor → Intento ruta Admin (403) → Logout Vendedor → Acceso dashboard sin sesión (redirect)
- **Componentes:** Rutas /login /logout /dashboard /productos. Middleware auth + role. AuthController. RoleMiddleware. User + Rol models. Vistas auth.login + dashboard.admin/vendedor.
- **Precondiciones:** BD tiene Rol 1=Admin, Rol 2=Vendedor. Usuarios "admin" (1/activo) pass Admin123; "vendedor" (2/activo) pass Vende123.
- **Pasos:** 1. GET /login → formulario. 2. POST login admin credenciales. 3. Ver redirect /dashboard admin (tarjetas "Usuarios activos", "Stock bajo"). 4. POST logout. 5. GET /login de nuevo. 6. POST login vendedor. 7. Dashboard VENDEDOR (sin Usuarios, sin Stock bajo). 8. URL manual /productos. 9. POST logout vendedor. 10. Manual /dashboard.
- **Resultado esperado:** Paso2→302 /dashboard; Paso3→dashboard admin 200; Paso4→redirect login; Paso6→OK; Paso7→dashboard vendedor; Paso8→HTTP 403 "No tienes permiso"; Paso9→logout OK; Paso10→redirect /login.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● ALTA
- **Evidencia:** 6 capturas
(dashboardAdmin/403/dashboardVendedor/intentoProducts/loginFinal/redirectSinSesion)

INT-002 Ciclo Producto → Stock → Venta (3 módulos)

- **ID:** INT-002
- **Flujo:** Admin crea Categoría (si hay UI, si no, existe por seed) → Admin crea Producto → Admin Entrada Stock → Vendedor consulta Stock (lectura OK, crear stock 403) → Vendedor registra Venta → Admin consulta Stock actualizado + Venta registrada
- **Componentes:** ProductoController, StockController, VentaController. Producto, Stock, MovimientoStock, Venta, DetalleVenta, TipoMovimiento, Categoria models. DB::transaction() venta, decrement stock. Vistas productos, stock, ventas.
- **Precondiciones:** Categoría id=1 existente (seed). TiposMovimiento: 1=Entrada, 2=Salida. Usuarios Admin y Vendedor creados.
- **Pasos:** 1. Login Admin → /productos/create → nombre "Cupcake Integral" precio=15 cat=1. 2. /stock/create → Entrada ×50, motivo "Inicial". 3. /stock → ver stock=50. 4. Logout Admin. 5. Login Vendedor. 6. /stock → ver listado (solo lectura). 7. Acceder /stock/create (debe fallar). 8. /ventas/create → Cupcake Integral ×3. 9. POST venta → redirect /ventas. 10. Logout Vendedor. 11. Login Admin → /stock (cupcake = 50-3 = 47) + /ventas (nueva venta total 15×3=45, usuario=vendedor).
- **Resultado esperado:** Stock final=47. Movimiento Entrada + Venta resta reflejada en decrement. Venta total=45. DetalleVenta 1 fila. Paso 7 → 403.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● ALTA
- **Evidencia:** SELECT stock, ventas, detalle_venta + capturas.

INT-003 Ciclo completo Pedido mixto + Estados + Pagos

- **ID:** INT-003
- **Flujo:** Vendedor registra Cliente → Crea Pedido mixto (productos + torta personalizada) → Actualiza estado Pendiente→En prep→Listo→Entregado → Registra Pago Parcial → Registra Pago Final hasta saldo 0
- **Componentes:** ClienteController, PedidoController (store, show, actualizarEstado, registrarPago). Cliente, Pedido, DetallePedido, DetalleTortaPersonalizada, EstadoPedido, Producto, User models. DB::transaction() Pedido store. Vistas clientes.create, pedidos.create/show.
- **Precondiciones:** Productos existentes: ProdA(50Bs), ProdB(30Bs). Estados 1..5. Usuario Vendedor logueado.
- **Pasos:** 1. /clientes/create → "PedidoIntegrador Test", carnet "PIT-001". 2. /pedidos/create. 3. Seleccionar cliente PIT-001, fecha_entrega = hoy +5 días, anticipo=20. 4. Productos: ProdA ×2 (=100), ProdB ×1 (=30). 5. Torta: porciones=15, sabor="Vainilla", precio=80, relleno="Crema". 6. POST pedido. 7. /pedidos/{id} ver detalle. 8. Actualizar estado=2 (En preparación). 9. Actualizar

estado=3 (Listo). 10. Registrar pago = 50 (parcial). 11. Actualizar estado=4 (Entregado). 12. Registrar pago = saldo restante.

- **Fórmulas:** Total = $(50 \times 2) + (30 \times 1) + 80 = 210$. Anticipo 20 → Saldo ini = 190. Pago 50 → Saldo 140. Pago final 140 → Saldo 0. pago_final = 190.
- **Resultado esperado:** Todo guardado. Pedidos: total=210, anticipo=20, pago_final=190, saldo=0. detalle_pedido: 2 filas. detalle_torta: 1 fila. Stock NO descontado.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● ALTA
- **Evidencia:** Captura detalle pedido + 3 SELECT (pedido / detalle / torta) + stock idéntico.

INT-004 Transaccionalidad Venta: producto excede → ROLLBACK total

- **ID:** INT-004
- **Flujo:** Venta con 3 productos: 2 OK, 1 excede stock. Debe hacer ROLLBACK COMPLETO (ninguna fila creada en ventas ni detalle_venta; stock intacto en los 3).
- **Componentes:** VentaController store DB::transaction(), lockForUpdate(). Venta, DetalleVenta, Stock, Producto models. MySQL InnoDB.
- **Precondiciones:** ProdA stock=5 precio=50; ProdB stock=3 precio=100; ProdB stock=10 precio=20. Login Usuario autorizado.
- **Pasos:** 1. /ventas/create. 2. Cantidades: ProdA=2 (OK), ProdB=4 (EXCEDE 3), ProdB=1 (OK). 3. POST enviar venta. 4. Después error: (a) SELECT COUNT() *ventas de hoy*; (b) SELECT COUNT() detalle_venta prod A,B,C; (c) SELECT stock A,B,C.
- **Resultado esperado:** UI: error "No hay suficiente stock para: ProdB". (a) 0 nuevas ventas (rollback). (b) 0 detalles nuevos. (c) Stock A=5, B=3, C=10 (sin cambios). Atomicidad.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● ALTA
- **Evidencia:** Captura error + 3 SELECT.

INT-005 Transaccionalidad Pedido: Anticipo > Total → ROLLBACK

- **ID:** INT-005
- **Flujo:** Pedido con productos OK, pero anticipo > total. Error sin guardar nada en pedidos/detalle/torta.
- **Componentes:** PedidoController store DB::transaction() con try/catch throw Exception.
- **Precondiciones:** Producto (25 Bs) stock=10. Cliente existente.
- **Pasos:** 1. /pedidos/create. 2. Producto × 3 → subtotal 75. 3. Anticipo = 200 (MAYOR que total). 4. POST. 5. SELECT COUNT(*) pedidos cliente_mismo; SELECT detalle_pedido; SELECT

torta_personalizada nuevo.

- **Resultado esperado:** Error UI "El anticipo no puede ser mayor que el total del pedido." 0 nuevos pedidos, 0 detalles (rollback transacción).
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● ALTA
- **Evidencia:** Captura error + SELECT.

INT-006 Gestión completa Usuario por Admin

- **ID:** INT-006
- **Flujo:** Admin crea Vendedor → Vendedor login → Admin Editar datos (sin password) → Vendedor login (contraseña antigua OK) → Admin Cambiar contraseña → Vendedor login (antigua falla, nueva OK) → Admin Desactivar → Vendedor login bloqueado
- **Componentes:** UsuarioController (store/edit/update/cambiarEstado). AuthController login. Hash::check + make. User/Rol models. Vistas usuarios.create/edit/index.
- **Precondiciones:** Login Admin activo.
- **Pasos:** 1. Admin crea usuario "Integracion Vendedor", UVINT001, usuario="vendedor_int", password="Primera123", rol=Vendedor. 2. Logout Admin; login vendedor_int. 3. Logout; login Admin. 4. Editar → nombres "Actualizado", tel "99999". Password vacío. 5. Logout Admin; login vendedor_int pass Primera123. 6. Admin edita password → "Segunda456" confirmación. 7. Vendedor login Primera123 (debe fallar) → Segunda456 (OK). 8. Admin → Listado → Desactivar vendedor_int. 9. Login vendedor_int → bloqueado.
- **Resultado esperado:** Cada paso se cumple. Hash password cambia solo si se introduce nueva.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● MEDIA
- **Evidencia:** 5 capturas + SELECT password antes/después.

INT-007 Dashboard Admin: métricas coinciden con BD real (semilla)

- **ID:** INT-007
- **Flujo:** Admin visualiza Dashboard; cada tarjeta se compara con query manual para confirmar cálculo de sum/count/whereDate.
- **Componentes:** DashboardController 6 consultas (ventasHoy sum, pedidosActivos count, usuariosActivos count, stockBajo count, pedidosHoy with, productosStockBajo with). Venta/Pedido/Stock/User models. Vista dashboard.admin.
- **Datos semilla (antes prueba):** 3 ventas HOY (100+50+30 = 180). 5 pedidos estado=1 + 1 est2 + 1 est3 = 7 activos. 2 usuarios activos/1 inactivo. 3 prod stock ≤5 (2, 3, 5). 2 pedidos

fecha_entrega=HOY.

- **Precondiciones:** Semillas insertadas. Login Admin.
- **Pasos:** 1. Login → /dashboard. 2. Comparar cada tarjeta con SQL manual individual.
- **Resultado esperado:** VentasHoy=180, pedidosActivos=7, usuariosActivos=2, stockBajo=3. Lista pedidosHoy 2 filas orden; productosStockBajo 3 filas (asc).
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● MEDIA
- **Evidencia:** Captura dashboard + cada query SQL individual.

INT-008 Movimientos Stock: Entrada → Salida → Ajuste

- **ID:** INT-008
- **Flujo:** Secuencia 3 movimientos; verifica stock final, 3 movimientos en histórico, usuario creador.
- **Componentes:** StockController store DB::transaction(). Stock, MovimientoStock, TipoMovimiento, Producto, User (Auth::id()).
- **Precondiciones:** Producto "MovTest" id=MP. Stock inicial 0. Tipos 1=Entrada, 2=Salida, 3=Ajuste. Admin id=1 logueado.
- **Pasos:** 1. /stock/create → Entrada MP ×100 motivo "Ingreso". 2. /stock → 100. 3. /stock/create → Salida ×30 motivo "Merma". 4. Stock → 70. 5. /stock/create → Ajuste ×200 motivo "Conteo". 6. Stock → 200. 7. SQL: SELECT * FROM movimientos_stock WHERE producto_id=MP ORDER BY created_at; SELECT stock.cantidad.
- **Resultado esperado:** 3 movimientos created_by=1, fecha≈now. Cantidad final 200.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● MEDIA
- **Evidencia:** 2 SELECT + 3 capturas movimientos.

INT-009 Validaciones múltiples simultáneas (formulario usuario)

- **ID:** INT-009
- **Flujo:** Formulario Usuario con múltiples errores al mismo tiempo; confirmar que Laravel retorna TODOS los errores en una sola respuesta, guarda cero registros.
- **Componentes:** UsuarioController store validate(): required, exists, max, unique, min, confirmed. Vista usuarios.create \$errors->all().
- **Precondiciones:** Usuario existente carnet="REPETIDO01", usuario="usuario_repe". Login Admin.
- **Pasos:** 1. /usuarios/create. 2. Rellenar: rol_id VACÍO. carnet=REPETIDO01. usuario=usuario_repe. password="123" (3). password_confirmation="456789" (6). 3. POST.

- **Resultado esperado:** Lista alerta ROJA con al menos 5 errores: (1) rol obligatorio, (2) carnet duplicado, (3) usuario duplicado, (4) password min 6, (5) password confirmación no coincide. BD NUEVA fila en usuarios.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● MEDIA
- **Evidencia:** Captura alerta errores + SELECT COUNT usuarios nueva fila = 0.

INT-010 Navegación completa UI (menú + Cancelar + Volver)

- **ID:** INT-010
- **Flujo:** Todos los enlaces del layout + botones Cancelar/Volver en todos los formularios responden correctamente (200, sin 404).
- **Componentes:** layouts.app blade. Todas vistas create de todos módulos. Todas rutas GET de web.php.
- **Precondiciones:** Login Admin (acceso completo).
- **Pasos:** 1. Dashboard → clic. 2. Productos → clic. 3. "Nuevo producto" → clic. 4. Cancelar → vuelve listado. 5. Stock → clic. 6. "Registrar movimiento" → create. 7. Cancelar → /stock. 8. Usuarios → index. 9. Nuevo → create. 10. Cancelar → /usuarios. 11. Ventas → index → Nueva → Cancelar → /ventas. 12. Pedidos → Nuevo → Cancelar → /pedidos. 13. Clientes → Nuevo → Cancelar → /clientes.
- **Resultado esperado:** TODOS HTTP 200. Cero 404. Cancelar siempre vuelve a ruta index del módulo.
- **Resultado obtenido:**
- **Estado:** PENDIENTE
- **Prioridad:** ● MEDIA
- **Evidencia:** Mapa de navegación o capturas por ruta.

Resumen: 10 flujos Integración

ID	Nombre	Prioridad
INT-001	Ciclo Auth completo + Roles Dashboard	● ALTA
INT-002	Producto → Stock → Venta	● ALTA
INT-003	Pedido completo mixto + pagos + estados	● ALTA
INT-004	Venta transaccional Rollback	● ALTA
INT-005	Pedido transaccional Rollback anticipo>total	● ALTA

ID	Nombre	Prioridad
INT-006	Gestión completa Usuario por Admin	● MEDIA
INT-007	Dashboard Admin cálculo métricas	● MEDIA
INT-008	Movimientos Stock Entrada→Salida→Ajuste	● MEDIA
INT-009	Validaciones múltiples formulario usuario	● MEDIA
INT-010	Navegación menú + Cancelar/Volver	● MEDIA